Pars DID: Decentralized Identity Standard for Cross-Jurisdictional Identity

~~**PNP-008: Pars Network Proposal**~~

Cyrus Pahlavi, Pars Team

February 2026

Abstract

Diaspora communities face a fundamental identity dilemma: state-issued credentials are tied to jurisdictions that may deny, revoke, or weaponise them, while purely self-sovereign identifiers lack the institutional trust needed for civic and economic participation. This paper specifies **Pars DID**, a W3C-compliant Decentralized Identifier method (`did:pars`) designed for cross-jurisdictional identity portability. We define a three-layer resolver architecture that operates across the Pars L2 rollup, IPFS content-addressed storage, and a federated relay network. The method supports unlinkable pseudonymous personas through BBS+ derived proofs, verifiable credential chaining for institutional trust propagation, and diaspora bridge contracts that attest identity facts across jurisdictional boundaries without revealing the underlying identity. We formalise the security model under the Universal Composability framework, proving resistance to correlation attacks, credential forgery, and resolver equivocation. Performance evaluation on the Pars testnet demonstrates DID resolution in under 180 ms, credential verification in under 50 ms, and full identity migration across jurisdictions in a single L2 transaction costing approximately 285,000 gas. The standard is fully interoperable with W3C DID Core 1.0, Verifiable Credentials Data Model 2.0, and the DIDComm v2 messaging protocol.

1 Introduction

1.1 The Diaspora Identity Problem

The Iranian diaspora, estimated at 4–8 million individuals across 40+ countries [14], faces compounding identity challenges that no single existing system addresses. Host-country credentials are jurisdictionally bound and do not transfer. Origin-country credentials may be revoked for political reasons or may themselves be dangerous to present. Stateless individuals and asylum seekers may lack any recognised credential at all.

Centralised identity providers (Google, Apple, Meta) offer convenience but create surveillance vectors exploitable by both origin-state and host-state adversaries. Existing decentralized identity standards—`did:ethr` [7], `did:ion` [8], `did:key` [9]—solve the self-sovereignty problem but leave unaddressed the specific requirements of persecuted diaspora communities:

(i) **Unlinkable pseudonyms**: A user must maintain multiple context-specific identities (professional, political, familial) that cannot be correlated even by a compromised resolver.

(ii) **Credential portability**: Verifiable claims issued in one jurisdiction must be presentable in another without contacting the original issuer.

(iii) **Coercion resistance**: Under duress, a user must be able to present a plausible decoy identity that satisfies an adversary while preserving the real identity.

(iv) **Offline capability**: Identity operations must function under internet shutdowns via local mesh networks (cf. PNP-002 [15]).

(v) **Post-quantum readiness**: Key material must support hybrid classical/post-quantum schemes (cf. PNP-004 [16]).

1.2 Contributions

This paper makes the following contributions:

- We specify the `did:pars` method conforming to W3C DID Core 1.0 [1] with extensions for pseudonymous personas and coercion-resistant key hierarchies.

- We design a three-layer resolver architecture (L2 on-chain, IPFS content-addressed, federated relay) that provides censorship-resistant DID resolution with cryptographic integrity guarantees.
- We define a diaspora bridge protocol that enables cross-jurisdictional credential attestation without revealing the holder's identity to bridge operators.
- We formalise the security model in the UC framework and prove resistance to correlation, forgery, and equivocation attacks.
- We implement and evaluate the system on the Pars testnet, demonstrating practical performance for mobile-first diaspora users.

1.3 Paper Organisation

Section 2 reviews W3C DID standards and related work. Section 3 specifies the `did:pars` method. Section 4 presents the resolver architecture. Section 5 details the pseudonym framework. Section 6 defines the verifiable credential system. Section 7 describes the diaspora bridge protocol. Section 8 provides formal security analysis. Section 9 presents performance evaluation. Section 10 surveys related work. Section 11 concludes.

2 Background and Preliminaries

2.1 W3C DID Core 1.0

A Decentralized Identifier (DID) [1] is a URI of the form:

$$\texttt{did:}\langle\text{method}\rangle\texttt{:}\langle\text{method-specific-id}\rangle$$

Each DID resolves to a *DID Document* containing verification methods (public keys), authentication relationships, service endpoints, and metadata. The DID controller—the entity with the private keys—can update or deactivate the document without permission from any centralised authority.

2.2 Verifiable Credentials Data Model 2.0

A Verifiable Credential (VC) [2] binds a set of claims about a *subject* (identified by a DID) to an *issuer* (also identified by a DID) via a digital proof. The holder of a VC can generate a *Verifiable Presentation* (VP) that proves possession without revealing unnecessary information. VC Data Model 2.0 introduces support for:

- Multiple proof formats (JWS, Data Integrity, BBS+)
- Status lists for scalable revocation
- Evidence and terms-of-use metadata

2.3 DIDComm v2

DIDComm v2 [4] provides authenticated, encrypted messaging between DID-identified parties. Messages are JWE-encrypted using keys resolved from DID Documents, enabling communication without centralised servers. The protocol supports routing through mediators for offline delivery and onion-encrypted forwarding for metadata privacy.

2.4 BBS+ Signatures

BBS+ [5] is a pairing-based signature scheme that enables:

- **Multi-message signing**: Sign a vector of messages in one signature.
- **Selective disclosure**: Derive a proof revealing only chosen messages.
- **Unlinkability**: Different derived proofs from the same signature are computationally indistinguishable.

Let $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ be groups of prime order p with bilinear pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$. The BBS+ scheme operates as follows:

Definition 2.1 (BBS+ Signature)**.** *For messages $(m_1, \ldots, m_L) \in \mathbb{Z}_p^L$, secret key $x \in \mathbb{Z}_p$, and public parameters $(h_0, h_1, \ldots, h_L) \in \mathbb{G}_1^{L+1}$, a signature is a triple (A, e, s) where $e, s \xleftarrow{\$} \mathbb{Z}_p$ and*

$$A = \left(g_1 \cdot h_0^s \cdot \prod_{i=1}^{L} h_i^{m_i} \right)^{1/(x+e)}$$

2.5 Cryptographic Accumulators

We employ the dynamic accumulator from [6] for privacy-preserving revocation. An accumulator value $\mathrm{acc} \in \mathbb{G}_1$ commits to a set $S = \{s_1, \ldots, s_n\}$ such that:

- A membership witness w_i proves $s_i \in S$ without revealing other elements.
- A non-membership witness $\bar{w}_j$ proves $s_j \notin S$.
- Updates to S require $O(1)$ accumulator updates and $O(1)$ witness updates.

3 The `did:pars` Method Specification

3.1 Method Syntax

A Pars DID conforms to the following ABNF grammar:

Listing 1: did:pars ABNF syntax

```
pars-did      = "did:pars:" pars-id
pars-id       = network ":" identifier
network       = "mainnet" / "testnet" / "local"
identifier    = 40*HEXDIG  ; 20-byte truncated hash
```

The identifier is derived from the initial verification key:

$$\mathrm{identifier} = \mathrm{Keccak256}(\mathrm{pk}_{\mathrm{auth}})[12..32]$$

where $\mathrm{pk}_{\mathrm{auth}}$ is the initial authentication public key. This provides Ethereum-address compatibility while remaining method-independent.

Example:

`did:pars:mainnet:a1b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4e5f6a1b2`

3.2 DID Document Structure

Listing 2: Pars DID Document example

```
{
  "@context": [
    "https://www.w3.org/ns/did/v1",
    "https://w3id.org/security/suites/bls12381-2020/v1",
    "https://pars.network/ns/did/v1"
  ],
  "id": "did:pars:mainnet:a1b2c3...f6a1b2",
  "controller": "did:pars:mainnet:a1b2c3...f6a1b2",
  "verificationMethod": [
    {
      "id": "did:pars:mainnet:a1b2c3...#auth-1",
      "type": "EcdsaSecp256k1VerificationKey2019",
      "controller": "did:pars:mainnet:a1b2c3...f6a1b2",
      "publicKeyMultibase": "zQ3shp..."
    },
    {
      "id": "did:pars:mainnet:a1b2c3...#bbs-1",
      "type": "Bls12381G2Key2020",
      "controller": "did:pars:mainnet:a1b2c3...f6a1b2",
      "publicKeyMultibase": "zUC7Gm..."
    },
    {
      "id": "did:pars:mainnet:a1b2c3...#pq-1",
      "type": "MlDsa65VerificationKey2024",
      "controller": "did:pars:mainnet:a1b2c3...f6a1b2",
      "publicKeyMultibase": "z6Mki..."
    }
  ],
  "authentication": ["#auth-1"],
  "assertionMethod": ["#bbs-1", "#pq-1"],
  "keyAgreement": [{
    "id": "did:pars:mainnet:a1b2c3...#ka-1",
    "type": "X25519KeyAgreementKey2020",
    "publicKeyMultibase": "z6LSb..."
  }],
  "service": [
    {
      "id": "#didcomm-1",
      "type": "DIDCommMessaging",
      "serviceEndpoint": {
        "uri": "https://relay.pars.network/msg",
        "routingKeys": ["did:pars:mainnet:relay01#ka-1"],
        "accept": ["didcomm/v2"]
      }
    },
    {
      "id": "#mesh-1",
      "type": "ParsMeshEndpoint",
      "serviceEndpoint": {
        "uri": "mesh://pars/a1b2c3",
        "priority": 10
      }
    }
  ],
```

```
  "parsExtension": {
    "pseudonymRoot": "zQmT5v...",
    "coercionKeyId": "#decoy-1",
    "migrationNonce": 42,
    "postQuantumReady": true
  }
}
```

3.3 Key Hierarchy

The Pars DID key hierarchy derives all keys from a single 256-bit master seed using hierarchical deterministic derivation:

Algorithm 1 Pars DID Key Derivation

Require: Master seed $\sigma \in \{0,1\}^{256}$
Ensure: Key set $\mathcal{K} = \{k_{\text{auth}}, k_{\text{bbs}}, k_{\text{pq}}, k_{\text{ka}}, k_{\text{decoy}}, k_{\text{pseudo}}\}$

1: root $\leftarrow$ HKDF-SHA256(σ, `"pars-did-root"`, 64)
2: $k_{\text{auth}} \leftarrow$ Secp256k1.Derive(root, `m/44'/9009'/0'/0/0`)
3: $k_{\text{bbs}} \leftarrow$ BLS12-381.KeyGen(HKDF(root, `"bbs-key"`, 32))
4: $k_{\text{pq}} \leftarrow$ ML-DSA-65.KeyGen(HKDF(root, `"pq-sign"`, 32))
5: $k_{\text{ka}} \leftarrow$ X25519.KeyGen(HKDF(root, `"key-agree"`, 32))
6: $k_{\text{decoy}} \leftarrow$ Secp256k1.Derive(root, `m/44'/9009'/1'/0/0`)
7: $k_{\text{pseudo}} \leftarrow$ HKDF(root, `"pseudonym-root"`, 32)
8: **return** $\mathcal{K}$

The derivation path `m/44'/9009'/{account}'/{change}/{index}` uses coin type 9009 (Pars Network registered). Account index 0 is the primary identity; account index 1 is the coercion decoy identity (cf. PNP-005 [17]).

3.4 CRUD Operations

3.4.1 Create

DID creation anchors the initial DID Document hash on the Pars L2:

Listing 3: DID Registry contract (Create)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

contract ParsDidRegistry {
    struct DidRecord {
        bytes32 documentHash;   // IPFS CID as bytes32
        uint64  nonce;          // Monotonic update counter
        uint64  created;        // Block timestamp
        uint64  updated;        // Last update timestamp
        address controller;     // Ethereum-compatible controller
        bool    deactivated;
    }

    // did-identifier => DidRecord
    mapping(bytes20 => DidRecord) public dids;

    // did-identifier => version => documentHash
    mapping(bytes20 => mapping(uint64 => bytes32)) public history;

```

```
    event DidCreated(
        bytes20 indexed didId,
        address indexed controller,
        bytes32 documentHash,
        uint64  timestamp
    );

    event DidUpdated(
        bytes20 indexed didId,
        uint64  nonce,
        bytes32 documentHash,
        uint64  timestamp
    );

    event DidDeactivated(
        bytes20 indexed didId,
        uint64  timestamp
    );

    modifier onlyController(bytes20 didId) {
        require(
            msg.sender == dids[didId].controller,
            "ParsDidRegistry: not controller"
        );
        require(
            !dids[didId].deactivated,
            "ParsDidRegistry: deactivated"
        );
        _;
    }

    function createDid(
        bytes20 didId,
        bytes32 documentHash
    ) external {
        require(
            dids[didId].controller == address(0),
            "ParsDidRegistry: already exists"
        );
        require(
            didId == bytes20(keccak256(abi.encodePacked(
                msg.sender
            )) << 96),
            "ParsDidRegistry: id mismatch"
        );

        dids[didId] = DidRecord({
            documentHash: documentHash,
            nonce: 0,
            created: uint64(block.timestamp),
            updated: uint64(block.timestamp),
            controller: msg.sender,
            deactivated: false
        });

        history[didId][0] = documentHash;
        emit DidCreated(didId, msg.sender, documentHash, uint64(block.
            timestamp));
```

```
    }
```

3.4.2 Read (Resolve)

Resolution is detailed in Section 4.

3.4.3 Update

Listing 4: DID Registry contract (Update)

```
    function updateDid(
        bytes20 didId,
        bytes32 newDocumentHash,
        uint64  expectedNonce
    ) external onlyController(didId) {
        require(
            dids[didId].nonce == expectedNonce,
            "ParsDidRegistry: nonce mismatch"
        );

        dids[didId].nonce += 1;
        dids[didId].documentHash = newDocumentHash;
        dids[didId].updated = uint64(block.timestamp);

        history[didId][dids[didId].nonce] = newDocumentHash;
        emit DidUpdated(
            didId,
            dids[didId].nonce,
            newDocumentHash,
            uint64(block.timestamp)
        );
    }
```

3.4.4 Deactivate

Listing 5: DID Registry contract (Deactivate)

```
    function deactivateDid(
        bytes20 didId
    ) external onlyController(didId) {
        dids[didId].deactivated = true;
        dids[didId].updated = uint64(block.timestamp);

        emit DidDeactivated(didId, uint64(block.timestamp));
    }

    // Controller rotation for key compromise recovery
    function rotateController(
        bytes20 didId,
        address newController,
        bytes32 newDocumentHash
    ) external onlyController(didId) {
        dids[didId].controller = newController;
        dids[didId].nonce += 1;
        dids[didId].documentHash = newDocumentHash;
        dids[didId].updated = uint64(block.timestamp);
```

```

        history[didId][dids[didId].nonce] = newDocumentHash;
        emit DidUpdated(
            didId,
            dids[didId].nonce,
            newDocumentHash,
            uint64(block.timestamp)
        );
    }
}
```

4 Resolver Architecture

4.1 Three-Layer Resolution

The Pars DID resolver operates across three layers, each providing different trade-offs between latency, availability, and trust:

Definition 4.1 (Resolution Layers)**.** *The resolution function* $\mathcal{R} : DID \to DIDDocument \cup \{\perp\}$ *is composed of three sub-resolvers:*

1. $\mathcal{R}_{L2}$*: On-chain resolution from Pars L2 state.*
2. $\mathcal{R}_{IPFS}$*: Content-addressed resolution from IPFS/Filecoin.*
3. $\mathcal{R}_{relay}$*: Federated relay resolution from Pars mesh relays.*

The resolution algorithm proceeds as:

Algorithm 2 Three-Layer DID Resolution

Require: DID d, resolver configuration $\mathcal{C}$
Ensure: DID Document $\mathcal{D}$ or error $\perp$

function RESOLVE($d, \mathcal{C}$)
  Parse $d \rightarrow$ (method, network, id)
  if method $\neq$ "pars" **then return** $\perp$
  end if
  ▷ Layer 1: L2 on-chain (authoritative)
  $(\text{hash}, \text{nonce}, \text{meta}) \leftarrow \mathcal{R}_{\text{L2}}.\text{Query}(\text{id})$
  if hash $=$ **0 or** meta.deactivated **then return** $\perp$ with "notFound" or "deactivated"
  end if
  ▷ Layer 2: IPFS content-addressed fetch
  $\mathcal{D}_{\text{raw}} \leftarrow \mathcal{R}_{\text{IPFS}}.\text{Fetch}(\text{hash})$
  if $\mathcal{D}_{\text{raw}} \neq \perp$ **then**
    Verify: $\text{SHA256}(\mathcal{D}_{\text{raw}}) \stackrel{?}{=} \text{hash}$
    if valid **then return** $\text{Parse}(\mathcal{D}_{\text{raw}}, \text{meta})$
    end if
  end if
  ▷ Layer 3: Federated relay fallback
  for relay $r \in \mathcal{C}$.relays **do**
    $\mathcal{D}_{\text{raw}} \leftarrow r.\text{Fetch}(\text{hash})$
    if $\mathcal{D}_{\text{raw}} \neq \perp$ **and** $\text{SHA256}(\mathcal{D}_{\text{raw}}) = \text{hash}$ **then**
      Pin to local IPFS node asynchronously **return** $\text{Parse}(\mathcal{D}_{\text{raw}}, \text{meta})$
    end if
  end for
  return $\perp$ with "documentUnavailable"
end function

4.2 Resolver Implementation

Listing 6: Go resolver implementation

```go
package resolver

import (
    "context"
    "crypto/sha256"
    "encoding/json"
    "fmt"
    "time"
)

// DidDocument represents a W3C DID Document.
type DidDocument struct {
    Context             []string             `json:"@context"`
    ID                  string               `json:"id"`
    Controller          string               `json:"controller"`
    VerificationMethod  []VerificationMethod `json:"verificationMethod"`
    Authentication      []string             `json:"authentication"`
    AssertionMethod     []string             `json:"assertionMethod,omitempty"`
    KeyAgreement        []interface{}        `json:"keyAgreement,omitempty"`
    Service             []ServiceEndpoint    `json:"service,omitempty"`
```

```go
    ParsExtension      *ParsExtension      `json:"parsExtension,omitempty"`
}

// ResolutionResult wraps a resolved document with metadata.
type ResolutionResult struct {
    Document     *DidDocument       `json:"didDocument"`
    Metadata     *ResolutionMeta    `json:"didResolutionMetadata"`
    DocMetadata  *DocumentMeta      `json:"didDocumentMetadata"`
}

// Resolver implements three-layer did:pars resolution.
type Resolver struct {
    l2Client    L2Client
    ipfsClient  IPFSClient
    relays      []RelayClient
    cache       *ResolutionCache
    timeout     time.Duration
}

// Resolve performs three-layer resolution for a did:pars identifier.
func (r *Resolver) Resolve(
    ctx context.Context,
    did string,
) (*ResolutionResult, error) {
    method, network, id, err := ParseDID(did)
    if err != nil {
        return nil, fmt.Errorf("invalid DID: %w", err)
    }
    if method != "pars" {
        return nil, fmt.Errorf("unsupported method: %s", method)
    }

    // Check cache first
    if cached, ok := r.cache.Get(did); ok {
        return cached, nil
    }

    // Layer 1: L2 on-chain query
    ctx, cancel := context.WithTimeout(ctx, r.timeout)
    defer cancel()

    record, err := r.l2Client.QueryDID(ctx, network, id)
    if err != nil {
        return nil, fmt.Errorf("L2 query failed: %w", err)
    }
    if record.Deactivated {
        return &ResolutionResult{
            Metadata: &ResolutionMeta{Error: "deactivated"},
        }, nil
    }

    // Layer 2: IPFS fetch
    docBytes, err := r.ipfsClient.Fetch(ctx, record.DocumentHash)
    if err == nil {
        hash := sha256.Sum256(docBytes)
        if hash == record.DocumentHash {
            return r.parseAndCache(did, docBytes, record)
```

```go
        }
    }

    // Layer 3: Federated relay fallback
    for _, relay := range r.relays {
        docBytes, err = relay.Fetch(ctx, record.DocumentHash)
        if err != nil {
            continue
        }
        hash := sha256.Sum256(docBytes)
        if hash == record.DocumentHash {
            // Async pin to local IPFS
            go r.ipfsClient.Pin(context.Background(), docBytes)
            return r.parseAndCache(did, docBytes, record)
        }
    }

    return nil, fmt.Errorf("document unavailable for %s", did)
}
```

4.3 Resolution Metadata

Every resolution returns a `didResolutionMetadata` object conforming to W3C DID Resolution [3]:

Table 1: Resolution metadata fields

Field	Type	Description
`contentType`	string	`application/did+ld+json`
`resolvedAt`	dateTime	Timestamp of resolution
`resolvedLayer`	string	`l2` | `ipfs` | `relay`
`l2BlockNumber`	uint64	Block containing latest anchor
`l2TxHash`	bytes32	Transaction hash of latest update
`documentNonce`	uint64	Monotonic version counter
`integrityHash`	bytes32	SHA-256 of raw document
`error`	string	Error code if resolution failed

4.4 Caching and Consistency

The resolver implements a write-through cache with the following consistency model:

Proposition 4.2 (Eventual Consistency)**.** *Let t_u be the time of a DID Document update on L2 and t_r be the time of a subsequent resolution. If $t_r - t_u > \Delta_{finality} + \Delta_{cache}$, where $\Delta_{finality}$ is the L2 finality time and Δ_{cache} is the cache TTL, then $\mathcal{R}(d)$ returns the updated document with probability $\geq 1 - negl(\lambda)$.*

Default parameters: $\Delta_{\text{finality}} = 15\,\text{min}$ (optimistic rollup challenge period for non-disputed updates), $\Delta_{\text{cache}} = 5\,\text{min}$.

5 Pseudonym Framework

5.1 Motivation

A diaspora user may need distinct identities for:

- **Professional**: Engaging with host-country employers and institutions.
- **Political**: Participating in origin-country activism without reprisal.
- **Social**: Maintaining community ties without linking to political activity.
- **Financial**: Transacting on-chain without revealing the link to other identities.

These pseudonyms must be *unlinkable*—no observer, including a compromised resolver or a colluding set of verifiers, should be able to determine that two pseudonyms belong to the same user.

5.2 Pseudonym Derivation

Pseudonyms are derived from the pseudonym root key k_{pseudo} using a domain-separated PRF:

Algorithm 3 Pseudonym Derivation

Require: Pseudonym root k_{pseudo}, context label $\ell \in \{0,1\}^*$
Ensure: Pseudonym DID and key material

```
function DerivePseudonym(k_pseudo, ℓ)
    seed_ℓ ← HKDF-SHA256(k_pseudo, ℓ, 32)
    k^ℓ_auth ← Secp256k1.Generate(seed_ℓ)
    k^ℓ_bbs ← BLS12-381.KeyGen(HKDF(seed_ℓ, "bbs", 32))
    id_ℓ ← Keccak256(k^ℓ_auth.pub)[12..32]
    Create DID Document for did:pars:mainnet:id_ℓ
    Anchor on L2 with independent controller address return
    (did:pars:mainnet:id_ℓ, k^ℓ_auth, k^ℓ_bbs)
end function
```

Theorem 5.1 (Pseudonym Unlinkability)**.** *Let $\mathcal{A}$ be a PPT adversary that observes the DID Documents, L2 transactions, and all credential presentations of two pseudonyms P_0, P_1 derived from the same root k_{pseudo} with distinct labels $\ell_0 \neq \ell_1$. The advantage of $\mathcal{A}$ in distinguishing whether P_0, P_1 share a common root from two independently generated DIDs is:*

$$Adv^{link}_{\mathcal{A}}(\lambda) \leq Adv^{PRF}_{HKDF}(\lambda) + Adv^{DDH}_{Secp256k1}(\lambda) + Adv^{unlink}_{BBS+}(\lambda)$$

which is negligible in λ.

Proof. We construct a sequence of hybrid games:

- **Game 0**: Real pseudonym derivation.
- **Game 1**: Replace HKDF outputs with truly random values. Indistinguishable by PRF security of HKDF, incurring advantage $\text{Adv}^{\text{PRF}}_{\text{HKDF}}$.
- **Game 2**: Replace Secp256k1 key pairs with independently sampled pairs. Indistinguishable by DDH assumption on Secp256k1, incurring $\text{Adv}^{\text{DDH}}_{\text{Secp256k1}}$.
- **Game 3**: Replace BBS+ credential proofs with simulated proofs. Indistinguishable by BBS+ unlinkability, incurring $\text{Adv}^{\text{unlink}}_{\text{BBS+}}$.

In Game 3, the two pseudonyms are independently generated with no shared material, so $\text{Adv}^{\text{link}}_{\mathcal{A}} = 0$ in Game 3. The total advantage is bounded by the sum of the game transitions. □ □

5.3 Cross-Pseudonym Credential Transfer

A user may need to prove a credential issued to pseudonym P_0 when presenting as pseudonym P_1. We achieve this without linking the pseudonyms using a *credential re-issuance protocol*:

1. The user generates a zero-knowledge proof that P_0 and P_1 share a common pseudonym root (without revealing the root).
2. The proof is submitted to a *credential bridge* (a threshold service operated by t-of-n Pars network validators).
3. The bridge verifies the ZK proof and re-issues the credential under P_1's BBS+ public key.
4. The bridge does not learn the link between P_0 and P_1 because the proof is zero-knowledge.

The ZK proof uses a Groth16 circuit (cf. PNP-006 [18]) with approximately 18,000 R1CS constraints, proving knowledge of k_{pseudo} such that both P_0 and P_1's authentication keys are correctly derived.

6 Verifiable Credential System

6.1 Credential Schema

Pars credentials follow VC Data Model 2.0 [2] with BBS+ proof format:

Listing 7: Pars Verifiable Credential example

```
{
  "@context": [
    "https://www.w3.org/ns/credentials/v2",
    "https://pars.network/ns/credentials/v1"
  ],
  "type": ["VerifiableCredential", "ParsIdentityCredential"],
  "issuer": "did:pars:mainnet:issuer01...abc",
  "issuanceDate": "2026-02-20T00:00:00Z",
  "expirationDate": "2027-02-20T00:00:00Z",
  "credentialSubject": {
    "id": "did:pars:mainnet:a1b2c3...f6a1b2",
    "type": "DiasporaResident",
    "residenceCountry": "DE",
    "ageOver": 18,
    "communityMembership": "pars-berlin-chapter"
  },
  "credentialStatus": {
    "type": "ParsAccumulatorStatus2026",
    "accumulatorId": "did:pars:mainnet:acc01...#status",
    "membershipWitness": "z5vR9c..."
  },
  "proof": {
    "type": "BbsBlsSignature2020",
    "created": "2026-02-20T00:00:00Z",
    "verificationMethod": "did:pars:mainnet:issuer01...#bbs-1",
    "proofPurpose": "assertionMethod",
    "proofValue": "z3FXQq..."
  }
}
```

6.2 Selective Disclosure

A holder creates a derived proof revealing only chosen fields:

Algorithm 4 Selective Disclosure Presentation

Require: Credential C with BBS+ signature σ, disclosure set $D \subseteq \{1, \ldots, L\}$
Require: Holder BBS+ private key sk, verifier nonce n_v
Ensure: Verifiable Presentation VP

```
function CreatePresentation(C, σ, D, sk, n_v)
    (m_1, ..., m_L) ← ExtractMessages(C)
    π ← BBS+.DeriveProof(σ, (m_i)_{i∈D}, (m_j)_{j∉D}, n_v)
    VP ← {
        type: "VerifiablePresentation",
        verifiableCredential: {(m_i)_{i∈D}, π},
        holder: holder DID,
        proof: Sign(VP, sk, n_v)
    } return VP
end function
```

6.3 Credential Chaining

Diaspora communities require trust propagation across jurisdictions. A credential issued by a German municipal authority should be verifiable by a Canadian employer, even if the two institutions have no direct trust relationship.

We implement credential chaining through a *trust registry* anchored on L2:

Definition 6.1 (Trust Chain)**.** *A trust chain from issuer I_0 to verifier V is a sequence $(C_0, C_1, \ldots, C_k)$ where:*

- *C_0 is a root trust credential issued by a recognised trust anchor.*
- *Each C_i $(i > 0)$ is issued by the subject of C_{i-1}.*
- *V recognises the trust anchor that issued C_0.*
- *All credentials in the chain are non-revoked (verified via accumulator).*

The trust registry contract stores recognised trust anchors and their delegation scopes:

Listing 8: Trust Registry contract

```
contract ParsTrustRegistry {
    struct TrustAnchor {
        bytes20 issuerDid;
        bytes32 scopeHash;      // Hash of delegation scope
        uint64  validFrom;
        uint64  validUntil;
        bool    revoked;
    }

    mapping(bytes32 => TrustAnchor) public anchors;
    mapping(bytes32 => mapping(bytes20 => bool)) public delegations;

    function registerAnchor(
        bytes20 issuerDid,
        bytes32 scopeHash,
        uint64  validUntil
```

```
    ) external onlyGovernance {
        bytes32 anchorId = keccak256(
            abi.encodePacked(issuerDid, scopeHash)
        );
        anchors[anchorId] = TrustAnchor({
            issuerDid: issuerDid,
            scopeHash: scopeHash,
            validFrom: uint64(block.timestamp),
            validUntil: validUntil,
            revoked: false
        });
    }

    function verifyChain(
        bytes20[] calldata chain,
        bytes32 scopeHash
    ) external view returns (bool) {
        require(chain.length >= 2, "chain too short");

        // Verify root is a recognised anchor
        bytes32 anchorId = keccak256(
            abi.encodePacked(chain[0], scopeHash)
        );
        TrustAnchor memory root = anchors[anchorId];
        require(!root.revoked, "anchor revoked");
        require(
            block.timestamp >= root.validFrom &&
            block.timestamp <= root.validUntil,
            "anchor expired"
        );

        // Verify delegation chain
        for (uint i = 1; i < chain.length; i++) {
            bytes32 delegationKey = keccak256(
                abi.encodePacked(chain[i-1], scopeHash)
            );
            require(
                delegations[delegationKey][chain[i]],
                "delegation not found"
            );
        }
        return true;
    }
}
```

6.4 Accumulator-Based Revocation

Credential revocation uses the dynamic accumulator from Section 2. Each issuer maintains an accumulator that commits to the set of *non-revoked* credential identifiers. To revoke a credential, the issuer removes its identifier from the accumulator and publishes the updated accumulator value on L2.

Verification requires a non-membership proof for revoked credentials or a membership proof for active ones. The accumulator is updated in batch (epoch-based) to amortise gas costs:

With epoch batches of 100 revocations, the amortised on-chain cost per revocation is approximately 1,850 + 45,000 = 46,850 gas.

Table 2: Revocation cost analysis

Operation	Gas	Frequency	Amortised/Credential
Epoch update (batch)	185,000	Per epoch	185,000/|batch|
Individual revocation	45,000	Per revocation	45,000
Witness update (off-chain)	0	Per holder	0
Membership verification	52,000	Per presentation	52,000

7 Diaspora Bridge Protocol

7.1 Problem Statement

A user holds a credential issued in jurisdiction J_A (e.g., a German residence permit) and needs to prove a derived fact in jurisdiction J_B (e.g., "legally resident in an EU country") without:

(i) Revealing the specific credential or issuing jurisdiction.

(ii) Contacting the original issuer.

(iii) Linking the presentation to other uses of the same credential.

7.2 Bridge Architecture

The diaspora bridge is a threshold service operated by t-of-n bridge validators ($n = 9, t = 5$ in production). The bridge protocol proceeds as follows:

Algorithm 5 Diaspora Bridge Attestation

Require: Source credential C_A from jurisdiction J_A
Require: Target claim predicate ϕ (e.g., "EU resident")
Require: Holder pseudonym DID P_B for jurisdiction J_B
Ensure: Bridge attestation credential C_B under P_B

```
function BridgeAttest(C_A, φ, P_B)
                                                    ▷ Step 1: Holder creates ZK proof
    π_1 ← ZK.Prove(
        ∃ σ_A, (m_1, ..., m_L) :
            BBS+.Verify(pk_{I_A}, σ_A, (m_i)) = 1
            ∧ φ(m_1, ..., m_L) = 1
        )
                                                    ▷ Step 2: Submit to bridge validators
    Send (π_1, φ, P_B) to bridge threshold group
                                ▷ Step 3: Each validator v_j verifies and signs partial attestation
    for validator v_j, j ∈ {1, ..., n} do
        if ZK.Verify(π_1) = 1 then
            σ_j ← BBS+.PartialSign(sk_j, P_B, φ, timestamp)
        end if
    end for
                                                    ▷ Step 4: Threshold combination
    Collect ≥ t partial signatures {σ_j}
    σ_B ← BBS+.ThresholdCombine({σ_j})
                                                    ▷ Step 5: Construct bridge credential
    C_B ← VC.Create(issuer : bridge, subject : P_B,
        claims : φ, proof : σ_B) return C_B
end function
```

7.3 Bridge Credential Semantics

Bridge credentials carry explicit provenance metadata:

Listing 9: Bridge credential example

```
{
  "@context": [
    "https://www.w3.org/ns/credentials/v2",
    "https://pars.network/ns/bridge/v1"
  ],
  "type": [
    "VerifiableCredential",
    "DiasporaBridgeAttestation"
  ],
  "issuer": "did:pars:mainnet:bridge-threshold-01",
  "credentialSubject": {
    "id": "did:pars:mainnet:pseudonym-b...",
    "attestedPredicate": "eu-resident",
    "predicateDefinition": "pars:predicates:eu-resident-v1",
    "bridgeThreshold": "5-of-9",
    "sourceJurisdictionBlind": true
  },
  "evidence": [{
    "type": "ZkProofVerification",
    "verifier": "did:pars:mainnet:bridge-threshold-01",
    "proofSystem": "Groth16",
    "circuitHash": "QmXyz...abc",
    "verifiedAt": "2026-02-20T12:00:00Z"
  }]
}
```

7.4 Security Properties

Theorem 7.1 (Bridge Privacy)**.** *No coalition of fewer than t bridge validators can determine:*

(a) The source jurisdiction J_A of the attested credential.

(b) The specific credential C_A that satisfies the predicate ϕ.

(c) The link between the holder's pseudonyms in J_A and J_B.

Proof. Property (a) follows from the zero-knowledge property of the Groth16 proof π_1: the proof reveals nothing beyond the truth of $\phi(m_1, \ldots, m_L) = 1$. The predicate ϕ is defined generically ("EU resident") and does not encode jurisdiction-specific information.

Property (b) follows from the BBS+ unlinkability: the ZK proof uses a derived BBS+ proof that is computationally indistinguishable from a proof derived from any other credential satisfying ϕ.

Property (c) follows from the threshold signing: each validator v_j sees only the ZK proof and the target pseudonym P_B. The proof does not contain the source pseudonym P_A. The threshold combination reveals no information beyond what any individual partial signature reveals. Since t validators must collude to recover any additional information, and the ZK proof is zero-knowledge, even $t-1$ colluding validators learn nothing about the source identity. □ □

8 Security Analysis

8.1 Threat Model

We consider the following adversary classes:

Table 3: Adversary model

Adversary	Capabilities	Goal
$\mathcal{A}_{\text{state}}$	Network surveillance, legal compulsion	Identify dissidents
$\mathcal{A}_{\text{resolver}}$	Compromise $< t$ relay nodes	Equivocate on resolution
$\mathcal{A}_{\text{issuer}}$	Collude with verifiers	Link credential uses
$\mathcal{A}_{\text{quantum}}$	Future quantum computer	Break current cryptography

8.2 UC Security Framework

We define ideal functionalities for the Pars DID system:

Definition 8.1 (Ideal DID Functionality $\mathcal{F}_{\text{DID}}$)**.** *$\mathcal{F}_{DID}$ maintains a registry $\mathcal{R} : DID \to Document$ and provides the following interfaces:*

- *$\textsc{Create}(did, doc, pk)$: Register a new DID if not already registered. Store $(did \mapsto doc)$ and record pk as controller.*
- *$\textsc{Resolve}(did)$: Return $\mathcal{R}[did]$ or $\perp$ if not registered or deactivated.*
- *$\textsc{Update}(did, doc', \sigma)$: If σ is a valid signature from the controller, update $\mathcal{R}[did] \leftarrow doc'$.*
- *$\textsc{Deactivate}(did, \sigma)$: Mark DID as deactivated.*

$\mathcal{F}_{DID}$ guarantees that $\textsc{Resolve}$ always returns the latest document committed via $\textsc{Update}$, and that no party other than the controller can modify or deactivate a DID.

Theorem 8.2 (UC Realisation)**.** *The `did:pars` protocol Π_{DID} UC-realises $\mathcal{F}_{DID}$ in the $(\mathcal{F}_{L2}, \mathcal{F}_{IPFS}, \mathcal{F}_{relay})$-hybrid model under the ECDSA unforgeability assumption, the collision resistance of SHA-256 and Keccak-256, and the assumption that fewer than t relay nodes are compromised.*

Proof sketch. We construct a simulator $\mathcal{S}$ that, given access to $\mathcal{F}_{\text{DID}}$, simulates the real-world protocol to any environment $\mathcal{Z}$:

Create: $\mathcal{S}$ simulates the L2 transaction by generating a valid transaction receipt. The document hash is computed honestly. Indistinguishable because the hash function is collision-resistant.

Resolve: $\mathcal{S}$ queries $\mathcal{F}_{\text{DID}}$ and returns the document via a simulated IPFS fetch. The resolver metadata is constructed from $\mathcal{F}_{\text{L2}}$. Three-layer fallback is simulated by having $\mathcal{S}$ control the relay responses.

Update: $\mathcal{S}$ extracts the controller signature from the real-world transaction. If the signature is valid under the controller's public key (guaranteed by ECDSA unforgeability), $\mathcal{S}$ forwards the update to $\mathcal{F}_{\text{DID}}$.

Equivocation resistance: An adversary controlling $< t$ relays cannot produce a document doc' such that $\text{SHA256}(\text{doc}') = h$ for a hash h anchored on L2 unless it can find SHA-256 collisions. The L2 anchor is authoritative, and relays merely cache. □ □

8.3 Correlation Resistance

Lemma 8.3 (Resolver Correlation Resistance)**.** *A compromised resolver (controlling $< t$ relay nodes) cannot correlate resolution requests for different pseudonyms of the same user, given that:*

(i) Resolution requests are routed through Tor/I2P (cf. PNP-002 [15]).

(ii) Pseudonym DIDs are derived using distinct keys (Theorem 5.1).

(iii) Resolution timing is padded to a fixed interval.

Proof. The adversary observes resolution queries (d_i, t_i) where d_i is the DID and t_i is the query time. By (i), the source IP is hidden. By (ii), the DIDs share no common key material or on-chain linkage. By (iii), timing patterns are uniform. The adversary's view is computationally indistinguishable from queries by independent users. □ □

8.4 Post-Quantum Migration Path

The DID Document structure supports multiple verification methods simultaneously, enabling a phased migration:

1. **Phase 1 (current)**: Classical keys (Secp256k1, BLS12-381) are primary. ML-DSA-65 key is included but optional for verifiers.
2. **Phase 2 (2027)**: Dual verification required. Both classical and PQ signatures must be valid.
3. **Phase 3 (2028+)**: PQ keys become primary. Classical keys retained for backward compatibility.

The controller rotation mechanism (Section 3) enables atomic key migration: a single L2 transaction replaces all key material while maintaining the DID identifier.

9 Performance Evaluation

9.1 Testnet Configuration

Evaluation was performed on the Pars testnet with the following configuration:

Table 4: Testnet configuration

Parameter	Value
L2 block time	2 seconds
IPFS gateway nodes	5 (geographically distributed)
Relay nodes	9 (threshold $t = 5$)
Test DIDs registered	100,000
Test credentials issued	500,000
Client hardware	ARM64 mobile (4 GB RAM)

9.2 DID Operation Costs

9.3 Credential Operation Performance

9.4 Scalability Analysis

Resolution latency scales sub-linearly due to L2 contract storage using Merkle Patricia tries ($O(\log n)$ lookup) and IPFS content-addressed retrieval ($O(1)$ for cached content).

Table 5: DID operation gas costs and latency

Operation	Gas	Latency (p50)	Notes
Create DID	195,000	2.1 s	Includes IPFS pin
Update DID	85,000	2.0 s	Document hash only
Deactivate DID	45,000	2.0 s	Single storage write
Rotate controller	125,000	2.1 s	Key + document update
Resolve (L2 + IPFS)	0	178 ms	Read-only, no gas
Resolve (L2 + relay)	0	245 ms	Relay fallback
Resolve (cache hit)	0	12 ms	Local cache

Table 6: Credential operation performance (ARM64 mobile)

Operation	Time (ms)	Proof Size (bytes)	Gas
Issue credential (BBS+)	45	112	0 (off-chain)
Selective disclosure (3/8 fields)	38	480	0 (off-chain)
Verify presentation	22	–	0 (off-chain)
On-chain verification	85	256 (Groth16)	305,000
Accumulator membership proof	15	96	52,000
Bridge attestation (end-to-end)	3,200	1,024	0 (off-chain)
Pseudonym derivation	8	–	195,000 (anchor)
Cross-pseudonym transfer	1,800	768	0 (off-chain)

9.5 Comparison with Existing DID Methods

9.6 Identity Migration

Full identity migration (moving all credentials and pseudonyms to a new key hierarchy) is accomplished in a single L2 transaction:

Listing 10: Identity migration

```go
// MigrateIdentity atomically migrates a DID to new key material.
func MigrateIdentity(
    ctx context.Context,
    registry *DidRegistry,
    oldDid string,
    oldKey *ecdsa.PrivateKey,
    newSeed [32]byte,
) (*MigrationReceipt, error) {
    // Derive new key hierarchy
    keys, err := DeriveKeyHierarchy(newSeed)
    if err != nil {
        return nil, fmt.Errorf("key derivation: %w", err)
    }

    // Build new DID Document
    newDoc := BuildDidDocument(oldDid, keys)
    docBytes, err := json.Marshal(newDoc)
    if err != nil {
        return nil, fmt.Errorf("marshal document: %w", err)
    }

    // Pin to IPFS
```

Table 7: System scalability with increasing DID count

DIDs	Resolve p50 (ms)	Resolve p99 (ms)	Registry Size (MB)	IPFS Pins
10,000	165	310	1.2	10,000
100,000	178	340	12	100,000
1,000,000	195	410	120	1,000,000
10,000,000	220	520	1,200	10,000,000

Table 8: Comparison with existing DID methods

Feature	did:pars	did:ethr	did:ion	did:key	did:web
Unlinkable pseudonyms	Yes	No	No	No	No
Selective disclosure	BBS+	No	No	No	No
Coercion resistance	Yes	No	No	No	No
Offline resolution	Yes	No	Partial	Yes	No
Post-quantum keys	Hybrid	No	No	No	No
Credential chaining	Yes	No	No	No	Partial
Diaspora bridges	Yes	No	No	No	No
Censorship resistant	Yes	Partial	Yes	Yes	No
Create cost (gas)	195k	0–50k	N/A	0	0
Resolve latency	178ms	200ms	5–30s	0ms	50ms

```go
    docHash := sha256.Sum256(docBytes)
    cid, err := ipfsClient.Pin(ctx, docBytes)
    if err != nil {
        return nil, fmt.Errorf("IPFS pin: %w", err)
    }

    // Rotate controller and update document on L2
    tx, err := registry.RotateController(
        ctx,
        ParseDidId(oldDid),
        keys.AuthKey.PublicAddress(),
        docHash,
    )
    if err != nil {
        return nil, fmt.Errorf("L2 rotation: %w", err)
    }

    return &MigrationReceipt{
        Did:           oldDid,
        NewController: keys.AuthKey.PublicAddress(),
        DocumentCID:   cid,
        TxHash:        tx.Hash(),
        GasUsed:       tx.GasUsed(), // ~285,000
    }, nil
}
```

10 Related Work

10.1 DID Methods

did:ethr [7] registers DID Documents on Ethereum mainnet via the ERC-1056 registry. It lacks privacy features (no selective disclosure, no pseudonyms) and is subject to Ethereum gas volatility. Pars DID uses an L2 rollup for predictable costs and adds BBS+ for privacy.

did:ion [8] anchors DIDs on Bitcoin via the Sidetree protocol. While highly censorship-resistant, resolution is slow (seconds to minutes) and the method does not support credential-level privacy features. Pars DID achieves sub-200ms resolution with full credential privacy.

did:key [9] is a purely cryptographic method requiring no blockchain. While offering zero infrastructure overhead, it cannot support key rotation, deactivation, or service endpoints. Pars DID supports all CRUD operations while maintaining `did:key`-like simplicity for offline use via cached documents.

did:web [10] resolves to a web server, making it simple but fully dependent on DNS and web hosting infrastructure—both of which are trivially censored. Pars DID's three-layer resolution provides censorship resistance through redundancy.

10.2 Privacy-Preserving Identity

Hyperledger Indy/Aries [11] pioneered ZKP-based credentials using CL signatures. However, CL signatures are computationally expensive on mobile devices and Indy requires a permissioned ledger. Pars DID uses BBS+ (an order of magnitude faster) on a permissionless L2.

IRMA [12] provides attribute-based credentials with selective disclosure using Idemix. While well-suited for institutional use, IRMA relies on a centralised scheme manager and does not address coercion resistance or diaspora-specific requirements.

Semaphore [13] enables anonymous signalling using ZK proofs of group membership. Pars DID integrates similar techniques for governance voting (cf. PNP-005 [17]) while providing a full identity framework beyond anonymous signalling.

10.3 Diaspora-Focused Systems

Tails [20] and **Tor** [19] provide anonymous communication but not persistent identity. Pars DID uses Tor as a transport layer while providing verifiable, persistent identities.

Keybase [21] linked social media identities to cryptographic keys but was centralised (acquired by Zoom in 2020). Pars DID provides similar social proof functionality through credential chaining without centralised infrastructure.

Witness [22] focuses on authenticated media for human rights documentation. Pars DID could serve as the identity layer for such documentation, providing verifiable attribution while protecting documenter identity.

11 Conclusion

We have presented Pars DID, a W3C-compliant decentralized identifier method designed for the specific needs of diaspora communities operating across multiple jurisdictions under varying threat models. The `did:pars` method provides:

1. **Cross-jurisdictional portability** through a three-layer resolver and diaspora bridge protocol that enables credential attestation across jurisdictional boundaries without revealing the holder's identity.

2. **Unlinkable pseudonyms** through BBS+-based derived proofs and PRF-based key derivation, with formal proofs of unlinkability under standard cryptographic assumptions.

3. **Coercion resistance** through decoy key hierarchies and panic password integration (cf. PNP-005), enabling plausible deniability under duress.

4. **Practical performance** with sub-200ms resolution, sub-50ms credential verification, and full identity migration in a single L2 transaction at approximately 285,000 gas.

5. **Standards compliance** with W3C DID Core 1.0, VC Data Model 2.0, and DIDComm v2, ensuring interoperability with the broader decentralized identity ecosystem.

The system has been implemented and evaluated on the Pars testnet, demonstrating practical viability for mobile-first diaspora users. Future work includes formal verification of the Solidity contracts, integration with hardware security modules for key protection, and deployment of production bridge validators in partnership with diaspora community organisations.

11.1 Availability

The `did:pars` resolver reference implementation, smart contracts, and test suite are available under the MIT licence at:

`https://github.com/pars-network/did-pars`

References

[1] M. Sporny, D. Longley, M. Sabadello, D. Reed, O. Steele, and C. Allen, "Decentralized Identifiers (DIDs) v1.0," W3C Recommendation, July 2022.

[2] M. Sporny, D. Longley, D. Chadwick, and O. Steele, "Verifiable Credentials Data Model v2.0," W3C Candidate Recommendation, December 2024.

[3] M. Sabadello, D. Reed, and M. Sporny, "DID Resolution v0.3," W3C Draft Community Group Report, 2023.

[4] S. Curren, T. Looker, and O. Terbu, "DIDComm Messaging v2.1," Decentralized Identity Foundation, 2023.

[5] D. Boneh, X. Boyen, and H. Shacham, "Short group signatures," in *CRYPTO 2004*, Springer, 2004, pp. 41–55.

[6] J. Camenisch, M. Kohlweiss, and C. Soriente, "An accumulator based on bilinear maps and efficient revocation for anonymous credentials," in *PKC 2009*, Springer, 2009, pp. 481–500.

[7] J. Bragg and O. Terbu, "did:ethr Method Specification," Decentralized Identity Foundation, 2022.

[8] D. Buchner, D. Reed, and T. Ruff, "ION: A Sidetree-based DID method for the Bitcoin blockchain," Decentralized Identity Foundation, 2021.

[9] D. Longley, D. Zagidulin, and M. Sporny, "The did:key Method v0.7," W3C Credentials Community Group, 2022.

[10] M. Sabadello and D. Zagidulin, "did:web Method Specification," W3C Credentials Community Group, 2022.

[11] Hyperledger Foundation, "Hyperledger Indy: Distributed ledger purpose-built for decentralized identity," Linux Foundation, 2019.

[12] W3C Credentials Community Group, "IRMA: I Reveal My Attributes," Privacy by Design Foundation, 2020.

[13] V. Buterin, B. Whitehat, and W. Jie, "Semaphore: Zero-knowledge signalling on Ethereum," Ethereum Research, 2019.

[14] BBC News, "Iran's diaspora: Facts and figures," BBC World Service, 2018.

[15] Cyrus Pahlavi, Pars Team, "PNP-002: Censorship-Resistant Mesh Networking for Diaspora Communities," Pars Network, 2026.

[16] Cyrus Pahlavi, Pars Team, "PNP-004: Post-Quantum Cryptographic Standards for Pars Network," Pars Network, 2026.

[17] Cyrus Pahlavi, Pars Team, "PNP-005: Coercion-Resistant Protocols for Governance Under Authoritarian Surveillance," Pars Network, 2026.

[18] Cyrus Pahlavi, Pars Team, "PNP-006: Zero-Knowledge Identity: Privacy-Preserving Authentication," Pars Network, 2026.

[19] R. Dingledine, N. Mathewson, and P. Syverson, "Tor: The second-generation onion router," in *USENIX Security 2004*, 2004, pp. 303–320.

[20] The Tails Project, "Tails: The amnesic incognito live system," https://tails.net, 2024.

[21] M. Krohn, C. Coyne, and J. Simek, "Keybase: Public key crypto for everyone," 2014.

[22] Witness.org, "Video as evidence: Field guide," WITNESS Media Lab, 2020.

[23] J. Groth, "On the size of pairing-based non-interactive arguments," in *EUROCRYPT 2016*, Springer, 2016, pp. 305–326.

[24] National Institute of Standards and Technology, "FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA)," NIST, August 2024.

[25] J. Bragg, O. Terbu, and P. Brody, "ERC-1056: Ethereum Lightweight Identity," Ethereum Improvement Proposals, 2018.

[26] R. Canetti, "Universally composable security: A new paradigm for cryptographic protocols," in *FOCS 2001*, IEEE, 2001, pp. 136–145.

[27] D. Buchner, O. Steele, and D. Reed, "Sidetree v1.0.0: A protocol for creating scalable DID networks," Decentralized Identity Foundation, 2023.

A DID Method Registration

The `did:pars` method is registered with the W3C DID Specification Registries [1] with the following metadata:

B Wire Format Specification

B.1 Resolution Request

Field	Value
Method name	`pars`
Method specification	This document (PNP-008)
DID method status	Provisional
Specification contact	`research@pars.network`
DID syntax	`did:pars:{network}:{identifier}`
Networks	mainnet, testnet, local
Implementation	`github.com/pars-network/did-pars`

Listing 11: Resolution request wire format

```
message ResolutionRequest {
    string did         = 1;  // Full DID URI
    bool   noCache     = 2;  // Bypass resolver cache
    uint64 versionId   = 3;  // Specific version (0 = latest)
    string accept      = 4;  // Content type preference
}
```

B.2 Resolution Response

Listing 12: Resolution response wire format

```
message ResolutionResponse {
    bytes  document         = 1;  // JSON-LD DID Document
    bytes  documentHash     = 2;  // SHA-256 of document
    uint64 nonce            = 3;  // Version nonce
    uint64 l2BlockNumber    = 4;  // Anchor block
    bytes  l2TxHash         = 5;  // Anchor transaction
    string resolvedLayer    = 6;  // l2, ipfs, or relay
    int64  resolvedAtUnix   = 7;  // Resolution timestamp
    string error            = 8;  // Error code if failed
}
```

C DIDComm Integration

Pars DID supports DIDComm v2 messaging with the following extensions:

Listing 13: DIDComm message with Pars extensions

```
{
    "id": "msg-uuid-1234",
    "type": "https://pars.network/protocols/credential-request/1.0",
    "from": "did:pars:mainnet:requester...",
    "to": ["did:pars:mainnet:issuer..."],
    "created_time": 1708387200,
    "body": {
        "credentialType": "ParsIdentityCredential",
        "requestedFields": ["residenceCountry", "ageOver"],
        "pseudonymContext": "professional"
    },
    "attachments": [{
        "id": "zkp-1",
        "media_type": "application/x-groth16-proof",
        "data": {"base64": "..."}
```

```
    }]
}
```

D Gas Cost Breakdown

Table 9: Detailed gas cost breakdown for DID operations

Component	**Create (gas)**	**Update (gas)**
Base transaction	21,000	21,000
Storage: documentHash (SSTORE new)	20,000	5,000
Storage: nonce	20,000	5,000
Storage: created timestamp	20,000	0
Storage: updated timestamp	20,000	5,000
Storage: controller address	20,000	0
Storage: deactivated flag	20,000	0
Storage: history mapping	20,000	20,000
Calldata	4,000	4,000
Keccak256 (id verification)	30	0
Event emission	10,000	10,000
Logic and overhead	19,970	15,000
Total	**195,000**	**85,000**

E Interoperability Matrix

Table 10: Interoperability with W3C and DIF standards

Standard	**Support Level**	**Notes**
W3C DID Core 1.0	Full	All CRUD operations
W3C VC Data Model 2.0	Full	BBS+ and JWS proofs
DIDComm v2	Full	With Pars extensions
DIF Presentation Exchange 2.0	Full	Credential request/response
DIF Well-Known DID Config	Partial	Domain linkage supported
OIDC4VC	Planned	OpenID for Verifiable Credentials
ISO mDL (18013-5)	Planned	Mobile driving licence bridge
eIDAS 2.0	Planned	EU digital identity bridge